

tmux documentation

basic-digital tmux — AGENTS.md

Base agent rules live at `constitution/AGENTS.md` and the `constitution/Constitution.md` it references. **READ THOSE FIRST.** This file extends them with project-specific rules; it never weakens them. Canonical project authority: [Constitution.md](#) — Project Articles §101–§109.

Fresh-conversation workflow: read `CONTINUATION.md` first (§0/§3/§8 mandatory), then `Issues.md` for `OPEN/PARTIAL/BLOCKED/RUNNING`.

MANDATORY ANTI-BLUFF END-USER-QUALITY COVENANT

Forensic anchor — verbatim user mandate (2026-04-28, reasserted 2026-05-21):

"We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This **MUST NOT** be the case and execution of tests and Challenges **MUST** guarantee the quality, the completion and full usability by end users of the product!"

Operative rule. The bar for shipping is **not** "tests pass" but **"end users can actually use the feature."** Every PASS — test OR HelixQA Challenge — **MUST** carry positive captured runtime evidence that the feature works for the end user. Metadata-only PASS, configuration- only PASS, "absence-of-error" PASS, and grep-without-runtime PASS are all critical defects regardless of how green the summary line looks. Tests and Challenges are bound **EQUALLY**.

This covenant is restated verbatim in every governance file at the consumer layer (`CLAUDE.md`, `AGENTS.md`, `QWEN.md`) so any tool that does not expand `@imports` still reads it. The same verbatim block lives upstream at `constitution/Constitution.md` §11.4, `constitution/CLAUDE.md`, and `constitution/AGENTS.md`. Removing or weakening this block is a §11.4-class release blocker.

Canonical authority: `constitution/Constitution.md` §11.4 and its sub-anchors §11.4.1 through §11.4.78. Project anchor: [Constitution.md](#) §101.

Non-compliance is a release blocker regardless of context.

Project overview

Verified hardened tmux 3.6a build with jemalloc, OOM protection, and per-session isolation via the tmx wrapper. Native dual-OS: each `tmx new -s NAME` spawns its own tmux server on the host with OS-native isolation — `cgroup-v2` scope on Linux, POSIX `rlimit` wrapper on macOS. Session shell is the operator's host shell with full `PATH`. No VM. macOS memory-cap gap documented honestly (XNU doesn't enforce `RLIMIT_AS`) — see `docs/guide/README.md` §5.6.

Critical base rules restated (for agents that don't follow @imports)

- **No bluffing.** Every PASS — test OR HelixQA Challenge — carries positive runtime evidence (file *content*, not existence). Universal §11.4 / Project §101.
- **FAIL-bluffs forbidden.** A test that exits FAIL for a script bug is fixed at the source layer, never at call sites. Universal §11.4.1.
- **No-guessing language.** likely, probably, maybe, seems, appears are forbidden when reporting causes — prove with forensic evidence or mark UNCONFIRMED: / PENDING_FORENSICS:. Universal §11.4.6.
- **Operator-path coverage.** Every gate test exercises the SAME entry point an end-user invokes (tmx new -s X). Project §102.
- **Four-layer coverage.** Every defect lands pre-build gate + runtime test + HelixQA Challenge + paired mutation. Project §103.
- **Use the commit wrapper.** bash commit_all.sh "<msg>" only — no direct git add / git commit / git push on the main repo.
- **Never force-push.** Explicit per-operation authorization required.
- **CONTINUATION.md** kept in sync in every non-trivial commit. Universal §12.10.
- **60 % RAM cap** enforced per-session via TMX_MEM. Project §106.

Commands (exact)

Step	Command
Install build deps	bash scripts/install_deps.sh (Linux: root; macOS: brew)
Full pipeline	bash scripts/setup.sh
Build only / Verify only	bash scripts/setup.sh --build-only / --verify-only
Verification gate	bash scripts/verify.sh
Run all tests	bash scripts/tests/run_all.sh
Single test	bash scripts/tests/NN_*.sh (01..17)
Inheritance gate	bash scripts/tests/ test_constitution_inheritance.sh
Meta-test (paired mutation)	bash scripts/tests/ meta_test_false_positive_proof.sh
End-to-end automation	bash scripts/test_e2e.sh
Commit + push (github+gitlab)	bash commit_all.sh "message"
Per-session wrapper	tmx {new attach ls kill}

Structure

Path	Role
constitution/	HelixConstitution submodule — universal governance — do not modify
tmux/	upstream submodule (tag 3.6a) — do not modify
Containers/	vasic-digital cgroup helpers submodule

Path	Role
<code>scripts/</code>	build, verify, install, 17 tests + inheritance gate, challenges, templates
<code>scripts/tmux</code>	generated dispatcher (<code>.gitignore'd</code> — edit <code>tmx.template</code>)
<code>scripts/tmux.conf.template</code>	tmux config template (scrollback / copy-mode / clipboard)
<code>commit_all.sh</code>	only allowed push mechanism
<code>Constitution.md</code>	project authority (extends <code>constitution/</code>)
<code>Issues.md / Fixed.md / CONTINUATION.md</code>	open work / closed archive / live handoff

Project-specific agent rules

Allowed CLI tools

- `git`, `gh`, `glab` (releases), `bash`, the project's own scripts.
- macOS: Homebrew (`brew`) for build deps. Linux: distro package manager via `install_deps.sh`.

Forbidden in this project

- Editing `tmux/`, `constitution/`, or generated files (`scripts/tmx`, `tmux/build*/`).
- Direct `git push` on the main repo — use `commit_all.sh`.
- CI/CD pipeline files (`.github/workflows/`, `.gitlab-ci.yml`).
- HTTPS Git remotes — SSH only.
- Advancing the `tmux/` submodule off tag 3.6a without an explicit documented decision (Project §107).

Project-specific workflow

- **Test-interrupt-on-discovery** (§103): a defect found mid-cycle STOPS the cycle — fix root cause, land four layers, rebuild, repeat.
- **Topology dispatch** (§104): tests detect host topology and SKIP-with-reason; never silently degrade.
- **Destructive tests** (12/13/14): require `TMX_TEST_DESTRUCTIVE=1`.
- **Releases**: bump `VERSION` (`version= + versionCode=` together) + `CHANGELOG.md` entry; `tag + gh release create + glab release create`.

Project overrides of universal rules

None.